

Bohloko Family Farm Poultry Processing System - SOLID Class Design

1. Introduction

This document presents a class design for the Bohloko Family Farm Poultry Processing System following SOLID principles:

- **S**ingle Responsibility Principle
- **O**pen/Closed Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation Principle
- **D**ependency Inversion Principle

2. Core Domain Entities

2.1 Base Entity Classes

```
// Base entity with common properties
abstract class Entity {
    protected id: string;
    protected createdAt: Date;
    protected updatedAt: Date;

    constructor(id?: string) {
        this.id = id || this.generateId();
        this.createdAt = new Date();
        this.updatedAt = new Date();
    }

    protected generateId(): string {
        return `${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
    }

    getId(): string {
        return this.id;
    }

    getCreatedAt(): Date {
        return this.createdAt;
    }

    getUpdatedAt(): Date {
        return this.updatedAt;
    }

    protected updateTimestamp(): void {
        this.updatedAt = new Date();
    }
}
```

```

    }
}

// Value object for money to handle currency operations
class Money {
    constructor(
        private amount: number,
        private currency: string = 'ZAR'
    ) {}

    getAmount(): number {
        return this.amount;
    }

    getCurrency(): string {
        return this.currency;
    }

    add(other: Money): Money {
        if (this.currency !== other.currency) {
            throw new Error('Cannot add different currencies');
        }
        return new Money(this.amount + other.amount, this.currency);
    }

    subtract(other: Money): Money {
        if (this.currency !== other.currency) {
            throw new Error('Cannot subtract different currencies');
        }
        return new Money(this.amount - other.amount, this.currency);
    }

    multiply(factor: number): Money {
        return new Money(this.amount * factor, this.currency);
    }

    equals(other: Money): boolean {
        return this.amount === other.amount && this.currency === other.currency;
    }
}

```

3. User Management Module

3.1 User Domain

```

// User role enumeration
enum UserRole {
    FARM_MANAGER = 'farm_manager',
    POULTRY_ATTENDANT = 'poultry_attendant',
    PROCESSING_STAFF = 'processing_staff',
}

```

```

    SALES_ASSISTANT = 'sales_assistant',
    CUSTOMER = 'customer'
}

// User type enumeration
enum UserType {
    CONSUMER = 'consumer',
    RESTAURANT = 'restaurant',
    RETAILER = 'retailer',
    DISTRIBUTOR = 'distributor',
    FARM_GATE = 'farm_gate',
    INSTITUTION = 'institution'
}

// Address value object
class Address {
    constructor(
        private street: string,
        private city: string,
        private state: string,
        private country: string,
        private postalCode: string
    ) {}

    getFullAddress(): string {
        return `${this.street}, ${this.city}, ${this.state} ${this.postalCode},
        ${this.country}`;
    }
}

// User profile value object
class UserProfile {
    constructor(
        private businessName?: string,
        private businessRegistrationNumber?: string,
        private taxId?: string,
        private address: Address,
        private contact: {
            phone: string;
            email: string;
        },
        private certifications?: {
            foodSafety?: string;
            halal?: boolean;
            organic?: boolean;
            other?: string[];
        }
    ) {}
}

// Base user class following SRP
abstract class User extends Entity {
    constructor(

```

```

    id: string,
    private email: string,
    private userType: UserType,
    private profile: UserProfile,
    private isActive: boolean = true
) {
    super(id);
}

getEmail(): string {
    return this.email;
}

getUserType(): UserType {
    return this.userType;
}

getProfile(): UserProfile {
    return this.profile;
}

isUserActive(): boolean {
    return this.isActive;
}

abstract getRole(): UserRole;

// OCP: Open for extension through inheritance
activate(): void {
    this.isActive = true;
    this.updateTimestamp();
}

deactivate(): void {
    this.isActive = false;
    this.updateTimestamp();
}
}

// Concrete user classes following LSP
class FarmStaffUser extends User {
    constructor(
        id: string,
        email: string,
        profile: UserProfile,
        private role: UserRole,
        private department: string
    ) {
        super(id, email, UserType.INSTITUTION, profile);
    }

    getRole(): UserRole {
        return this.role;
    }
}

```

```

    }

    getDepartment(): string {
        return this.department;
    }
}

class CustomerUser extends User {
    constructor(
        id: string,
        email: string,
        userType: UserType,
        profile: UserProfile,
        private loyaltyPoints: number = 0
    ) {
        super(id, email, userType, profile);
    }

    getRole(): UserRole {
        return UserRole.CUSTOMER;
    }

    getLoyaltyPoints(): number {
        return this.loyaltyPoints;
    }

    addLoyaltyPoints(points: number): void {
        this.loyaltyPoints += points;
        this.updateTimestamp();
    }
}

```

3.2 Authentication Services

```

// Interface for authentication following ISP
interface Authenticator {
    authenticate(credentials: any): Promise<string>;
    validateToken(token: string): Promise<boolean>;
}

interface PasswordHasher {
    hash(password: string): Promise<string>;
    verify(password: string, hash: string): Promise<boolean>;
}

// Concrete implementations
class JwtAuthenticator implements Authenticator {
    async authenticate(credentials: any): Promise<string> {
        // JWT implementation
        return 'jwt-token';
    }
}

```

```

    }

    async validateToken(token: string): Promise<boolean> {
        // Token validation logic
        return true;
    }
}

class BcryptPasswordHasher implements PasswordHasher {
    async hash(password: string): Promise<string> {
        // bcrypt implementation
        return `hashed-${password}`;
    }

    async verify(password: string, hash: string): Promise<boolean> {
        // bcrypt verification
        return hash === `hashed-${password}`;
    }
}

// User service following DIP
class UserService {
    constructor(
        private userRepository: UserRepository,
        private authenticator: Authenticator,
        private passwordHasher: PasswordHasher
    ) {}

    async registerUser(userData: any): Promise<User> {
        // Registration logic
        return new CustomerUser('id', 'email', UserType.CONSUMER, {} as
UserProfile);
    }

    async authenticateUser(email: string, password: string): Promise<string> {
        // Authentication logic
        return this.authenticator.authenticate({ email, password });
    }
}

// Repository interface
interface UserRepository {
    findById(id: string): Promise<User | null>;
    findByEmail(email: string): Promise<User | null>;
    save(user: User): Promise<void>;
    delete(id: string): Promise<void>;
}

```

4. Production Management Module

4.1 Production Domain

```

// Production type enumeration
enum ProductionType {
    BROILER_CYCLE = 'broiler_cycle',
    EGG_PRODUCTION = 'egg_production',
    HATCHING = 'hatching'
}

// Production status enumeration
enum ProductionStatus {
    PLANNED = 'planned',
    IN_PROGRESS = 'in_progress',
    COMPLETED = 'completed',
    CANCELLED = 'cancelled'
}

// Feed type enumeration
enum FeedType {
    STARTER = 'starter',
    GROWER = 'grower',
    FINISHER = 'finisher',
    LAYER = 'layer'
}

// Production batch value object
class ProductionBatch {
    constructor(
        private batchNumber: string,
        private birdCount: number,
        private startDate: Date,
        private expectedHarvestDate: Date
    ) {}

    getAgeInDays(): number {
        const diff = new Date().getTime() - this.startDate.getTime();
        return Math.floor(diff / (1000 * 3600 * 24));
    }

    isReadyForHarvest(): boolean {
        return new Date() >= this.expectedHarvestDate;
    }
}

// Production cycle entity
class ProductionCycle extends Entity {
    private batches: ProductionBatch[] = [];
    private status: ProductionStatus = ProductionStatus.PLANNED;
    private dailyLogs: DailyProductionLog[] = [];

    constructor(
        id: string,
        private cycleNumber: string,

```

```

    private type: ProductionType,
    private expectedDuration: number, // days
    private farmManagerId: string
  ) {
    super(id);
  }

  addBatch(batch: ProductionBatch): void {
    this.batches.push(batch);
    this.updateTimestamp();
  }

  startCycle(): void {
    if (this.status !== ProductionStatus.PLANNED) {
      throw new Error('Cycle can only be started from planned status');
    }
    this.status = ProductionStatus.IN_PROGRESS;
    this.updateTimestamp();
  }

  completeCycle(): void {
    this.status = ProductionStatus.COMPLETED;
    this.updateTimestamp();
  }

  addDailyLog(log: DailyProductionLog): void {
    this.dailyLogs.push(log);
    this.updateTimestamp();
  }

  getTotalBirds(): number {
    return this.batches.reduce((total, batch) => total + batch.birdCount, 0);
  }

  getStatus(): ProductionStatus {
    return this.status;
  }
}

// Daily production log value object
class DailyProductionLog {
  constructor(
    private date: Date,
    private birdCount: number,
    private feedConsumed: number,
    private waterConsumed: number,
    private mortalityCount: number,
    private temperature: { min: number; max: number },
    private recordedBy: string
  ) {}

  calculateMortalityRate(totalBirds: number): number {
    if (totalBirds === 0) return 0;
  }
}

```



```

        return (this.mortalityCount / totalBirds) * 100;
    }
}

// Medication record
class MedicationRecord {
    constructor(
        private medicationName: string,
        private dosage: string,
        private administrationDate: Date,
        private administeredBy: string,
        private notes?: string
    ) {}
}

```

4.2 Production Services

```

// Interface for production operations
interface ProductionOperations {
    planCycle(plan: ProductionPlan): Promise<ProductionCycle>;
    startCycle(cycleId: string): Promise<void>;
    recordDailyLog(cycleId: string, log: DailyProductionLog): Promise<void>;
    administerMedication(cycleId: string, medication: MedicationRecord):
    Promise<void>;
}

// Production plan value object
class ProductionPlan {
    constructor(
        private expectedBirds: number,
        private expectedDuration: number,
        private budget: Money,
        private startDate: Date
    ) {}

    calculateDailyCost(): Money {
        return this.budget.multiply(1 / this.expectedDuration);
    }
}

// Production service
class ProductionService implements ProductionOperations {
    constructor(
        private productionRepository: ProductionRepository,
        private notificationService: NotificationService
    ) {}

    async planCycle(plan: ProductionPlan): Promise<ProductionCycle> {
        const cycle = new ProductionCycle(
            'cycle-id',

```

```

        `CYC-${Date.now()}`,
        ProductionType.BROILER_CYCLE,
        plan.expectedDuration,
        'manager-id'
    );

    await this.productionRepository.save(cycle);
    await this.notificationService.notifyFarmManager('New production cycle
planned');

    return cycle;
}

async startCycle(cycleId: string): Promise<void> {
    const cycle = await this.productionRepository.findById(cycleId);
    if (!cycle) throw new Error('Cycle not found');

    cycle.startCycle();
    await this.productionRepository.save(cycle);
}

async recordDailyLog(cycleId: string, log: DailyProductionLog): Promise<void>
{
    const cycle = await this.productionRepository.findById(cycleId);
    if (!cycle) throw new Error('Cycle not found');

    cycle.addDailyLog(log);

    // Check for high mortality alert
    const mortalityRate = log.calculateMortalityRate(cycle.getTotalBirds());
    if (mortalityRate > 5) {
        await this.notificationService.sendAlert(
            'High mortality rate detected',
            `Mortality rate: ${mortalityRate.toFixed(2)}%`
        );
    }

    await this.productionRepository.save(cycle);
}

async administerMedication(cycleId: string, medication: MedicationRecord):
Promise<void> {
    // Medication administration logic
    await this.productionRepository.addMedication(cycleId, medication);
}

interface ProductionRepository {
    findById(id: string): Promise<ProductionCycle | null>;
    save(cycle: ProductionCycle): Promise<void>;
    addMedication(cycleId: string, medication: MedicationRecord): Promise<void>;
}

```

5. Inventory Management Module

5.1 Inventory Domain

```
// Product type enumeration
enum ProductType {
    WHOLE_CHICKEN = 'whole_chicken',
    BREAST = 'breast',
    THIGHS = 'thighs',
    WINGS = 'wings',
    DRUMSTICKS = 'drumsticks',
    GIZZARDS = 'gizzards',
    FEET = 'feet'
}

// Storage location
class StorageLocation {
    constructor(
        private locationId: string,
        private name: string,
        private temperature: number, // Celsius
        private capacity: number,
        private currentStock: number = 0
    ) {}

    getAvailableCapacity(): number {
        return this.capacity - this.currentStock;
    }

    canStore(quantity: number): boolean {
        return this.getAvailableCapacity() >= quantity;
    }

    addStock(quantity: number): void {
        if (!this.canStore(quantity)) {
            throw new Error('Insufficient storage capacity');
        }
        this.currentStock += quantity;
    }

    removeStock(quantity: number): void {
        if (this.currentStock < quantity) {
            throw new Error('Insufficient stock');
        }
        this.currentStock -= quantity;
    }
}

// Inventory item
class InventoryItem extends Entity {
```

```

constructor(
    id: string,
    private productType: ProductType,
    private batchNumber: string,
    private quantity: number,
    private unitWeight: number, // kg
    private storageLocation: StorageLocation,
    private harvestDate: Date,
    private expiryDate: Date,
    private unitCost: Money
) {
    super(id);
}

getTotalWeight(): number {
    return this.quantity * this.unitWeight;
}

getTotalCost(): Money {
    return this.unitCost.multiply(this.quantity);
}

isExpired(): boolean {
    return new Date() > this.expiryDate;
}

daysUntilExpiry(): number {
    const diff = this.expiryDate.getTime() - new Date().getTime();
    return Math.floor(diff / (1000 * 3600 * 24));
}

split(quantity: number): InventoryItem {
    if (quantity > this.quantity) {
        throw new Error('Cannot split more than available quantity');
    }

    this.quantity -= quantity;
    this.updateTimestamp();

    return new InventoryItem(
        this.generateId(),
        this.productType,
        this.batchNumber,
        quantity,
        this.unitWeight,
        this.storageLocation,
        this.harvestDate,
        this.expiryDate,
        this.unitCost
    );
}
}

```

```
// Inventory transaction
class InventoryTransaction {
  constructor(
    private transactionId: string,
    private itemId: string,
    private transactionType: 'IN' | 'OUT' | 'ADJUSTMENT',
    private quantity: number,
    private reason: string,
    private performedBy: string,
    private timestamp: Date = new Date()
  ) {}
}
```

5.2 Inventory Services

```
// Inventory operations interface
interface InventoryOperations {
  addItem(item: InventoryItem): Promise<void>;
  removeItem(itemId: string, quantity: number, reason: string): Promise<void>;
  transferItem(itemId: string, newLocation: StorageLocation, quantity: number):
  Promise<void>;
  getStockLevel(productType: ProductType): Promise<number>;
}

// Inventory service
class InventoryService implements InventoryOperations {
  constructor(
    private inventoryRepository: InventoryRepository,
    private transactionLogger: TransactionLogger
  ) {}

  async addItem(item: InventoryItem): Promise<void> {
    // Check storage capacity
    const location = item.getStorageLocation();
    if (!location.canStore(item.getQuantity())) {
      throw new Error('Insufficient storage capacity');
    }

    // Add to inventory
    await this.inventoryRepository.save(item);

    // Log transaction
    await this.transactionLogger.logTransaction(
      new InventoryTransaction(
        'txn-id',
        item.getId(),
        'IN',
        item.getQuantity(),
        'Harvest processing',
        'system'
      )
    );
  }
}
```

```

    )
  );

  // Update storage location
  location.addStock(item.getQuantity());
}

async removeItem(itemId: string, quantity: number, reason: string):
Promise<void> {
  const item = await this.inventoryRepository.findById(itemId);
  if (!item) throw new Error('Item not found');

  if (item.getQuantity() < quantity) {
    throw new Error('Insufficient stock');
  }

  // Create split item for removal
  const removedItem = item.split(quantity);

  // Log transaction
  await this.transactionLogger.logTransaction(
    new InventoryTransaction(
      'txn-id',
      itemId,
      'OUT',
      quantity,
      reason,
      'system'
    )
  );

  await this.inventoryRepository.save(item);
}

async transferItem(itemId: string, newLocation: StorageLocation, quantity:
number): Promise<void> {
  const item = await this.inventoryRepository.findById(itemId);
  if (!item) throw new Error('Item not found');

  if (item.getQuantity() < quantity) {
    throw new Error('Insufficient stock');
  }

  // Check if new location has capacity
  if (!newLocation.canStore(quantity)) {
    throw new Error('New location has insufficient capacity');
  }

  // Split and transfer
  const transferredItem = item.split(quantity);
  transferredItem.updateStorageLocation(newLocation);

  // Update locations

```

```

        item.getStorageLocation().removeStock(quantity);
        newLocation.addStock(quantity);

        // Log transaction
        await this.transactionLogger.logTransaction(
            new InventoryTransaction(
                'txn-id',
                itemId,
                'TRANSFER',
                quantity,
                `Transferred to ${newLocation.getName()}`,
                'system'
            )
        );

        await this.inventoryRepository.save(item);
        await this.inventoryRepository.save(transferredItem);
    }

    async getStockLevel(productType: ProductType): Promise<number> {
        const items = await
this.inventoryRepository.findByProductType(productType);
        return items.reduce((total, item) => total + item.getQuantity(), 0);
    }
}

interface InventoryRepository {
    findById(id: string): Promise<InventoryItem | null>;
    findByProductType(productType: ProductType): Promise<InventoryItem[]>;
    save(item: InventoryItem): Promise<void>;
    delete(id: string): Promise<void>;
}

interface TransactionLogger {
    logTransaction(transaction: InventoryTransaction): Promise<void>;
    getTransactionHistory(itemId: string): Promise<InventoryTransaction[]>;
}

```

6. Order Management Module

6.1 Order Domain

```

// Order status enumeration
enum OrderStatus {
    PENDING = 'pending',
    CONFIRMED = 'confirmed',
    PROCESSING = 'processing',
    SHIPPED = 'shipped',
    DELIVERED = 'delivered',
    CANCELLED = 'cancelled'
}

```

```

}

// Payment method enumeration
enum PaymentMethod {
    CREDIT_CARD = 'credit_card',
    DEBIT_CARD = 'debit_card',
    BANK_TRANSFER = 'bank_transfer',
    MOBILE_MONEY = 'mobile_money',
    CASH_ON_DELIVERY = 'cash_on_delivery'
}

// Shipping method enumeration
enum ShippingMethod {
    STANDARD = 'standard',
    EXPRESS = 'express',
    PICKUP = 'pickup',
    FARM_GATE = 'farm_gate',
    LOCAL_DELIVERY = 'local_delivery'
}

// Order item value object
class OrderItem {
    constructor(
        private productId: string,
        private productName: string,
        private quantity: number,
        private unitPrice: Money,
        private batchNumber?: string
    ) {}

    getTotalPrice(): Money {
        return this.unitPrice.multiply(this.quantity);
    }
}

// Order entity
class Order extends Entity {
    private items: OrderItem[] = [];
    private status: OrderStatus = OrderStatus.PENDING;

    constructor(
        id: string,
        private orderNumber: string,
        private customerId: string,
        private shippingAddress: Address,
        private billingAddress: Address,
        private paymentMethod: PaymentMethod,
        private shippingMethod: ShippingMethod
    ) {
        super(id);
    }

    addItem(item: OrderItem): void {

```



```

        this.items.push(item);
        this.updateTimestamp();
    }

    removeItem(productId: string): void {
        this.items = this.items.filter(item => item.getProductId() !== productId);
        this.updateTimestamp();
    }

    confirm(): void {
        if (this.status !== OrderStatus.PENDING) {
            throw new Error('Order can only be confirmed from pending status');
        }
        this.status = OrderStatus.CONFIRMED;
        this.updateTimestamp();
    }

    cancel(reason: string): void {
        if (this.status === OrderStatus.SHIPPED || this.status ===
OrderStatus.DELIVERED) {
            throw new Error('Cannot cancel shipped or delivered orders');
        }
        this.status = OrderStatus.CANCELLED;
        this.updateTimestamp();
    }

    getTotalAmount(): Money {
        return this.items.reduce((total, item) => total.add(item.getTotalPrice()),
new Money(0));
    }

    getItemCount(): number {
        return this.items.length;
    }

    getStatus(): OrderStatus {
        return this.status;
    }
}

// Payment entity
class Payment extends Entity {
    constructor(
        id: string,
        private orderId: string,
        private amount: Money,
        private method: PaymentMethod,
        private status: 'pending' | 'paid' | 'failed' | 'refunded' = 'pending'
    ) {
        super(id);
    }

    process(): void {

```

```

    // Payment processing logic
    this.status = 'paid';
    this.updateTimestamp();
  }

  refund(reason: string): void {
    this.status = 'refunded';
    this.updateTimestamp();
  }

  getStatus(): string {
    return this.status;
  }
}

```

6.2 Order Services

```

// Order operations interface
interface OrderOperations {
  createOrder(orderData: any): Promise<Order>;
  confirmOrder(orderId: string): Promise<void>;
  cancelOrder(orderId: string, reason: string): Promise<void>;
  processPayment(orderId: string, paymentData: any): Promise<Payment>;
}

// Order service
class OrderService implements OrderOperations {
  constructor(
    private orderRepository: OrderRepository,
    private inventoryService: InventoryService,
    private paymentProcessor: PaymentProcessor,
    private notificationService: NotificationService
  ) {}

  async createOrder(orderData: any): Promise<Order> {
    // Validate inventory availability
    for (const item of orderData.items) {
      const stockLevel = await
this.inventoryService.getStockLevel(item.productType);
      if (stockLevel < item.quantity) {
        throw new Error(`Insufficient stock for ${item.productName}`);
      }
    }

    // Create order
    const order = new Order(
      'order-id',
      `ORD-${Date.now()}`,
      orderData.customerId,
      orderData.shippingAddress,

```

```

        orderData.billingAddress,
        orderData.paymentMethod,
        orderData.shippingMethod
    );

    // Add items
    for (const itemData of orderData.items) {
        order.addItem(new OrderItem(
            itemData.productId,
            itemData.productName,
            itemData.quantity,
            new Money(itemData.unitPrice),
            itemData.batchNumber
        ));
    }

    // Reserve inventory
    for (const item of orderData.items) {
        await this.inventoryService.reserveItem(item.productId, item.quantity,
`Order ${order.getOrderNumber()}`);
    }

    await this.orderRepository.save(order);
    await this.notificationService.sendOrderConfirmation(order);

    return order;
}

async confirmOrder(orderId: string): Promise<void> {
    const order = await this.orderRepository.findById(orderId);
    if (!order) throw new Error('Order not found');

    order.confirm();
    await this.orderRepository.save(order);

    // Notify processing staff
    await this.notificationService.notifyProcessingStaff(`New order confirmed:
${order.getOrderNumber()}`);
}

async cancelOrder(orderId: string, reason: string): Promise<void> {
    const order = await this.orderRepository.findById(orderId);
    if (!order) throw new Error('Order not found');

    order.cancel(reason);

    // Release reserved inventory
    for (const item of order.getItems()) {
        await this.inventoryService.releaseReservation(item.getProductId(),
item.getQuantity());
    }

    await this.orderRepository.save(order);
}

```

```

    await this.notificationService.sendOrderCancellation(order, reason);
  }

  async processPayment(orderId: string, paymentData: any): Promise<Payment> {
    const order = await this.orderRepository.findById(orderId);
    if (!order) throw new Error('Order not found');

    const payment = new Payment(
      'payment-id',
      orderId,
      order.getTotalAmount(),
      order.getPaymentMethod()
    );

    // Process payment
    await this.paymentProcessor.process(payment, paymentData);

    if (payment.getStatus() === 'paid') {
      await this.confirmOrder(orderId);
    }

    await this.orderRepository.savePayment(payment);
    return payment;
  }
}

interface OrderRepository {
  findById(id: string): Promise<Order | null>;
  save(order: Order): Promise<void>;
  savePayment(payment: Payment): Promise<void>;
}

interface PaymentProcessor {
  process(payment: Payment, paymentData: any): Promise<void>;
  refund(payment: Payment, reason: string): Promise<void>;
}

```

7. Analytics & Reporting Module

7.1 Analytics Domain

```

// Analytics interface following ISP
interface ProductionAnalytics {
  calculateMortalityRate(cycleId: string): Promise<number>;
  calculateFeedConversionRatio(cycleId: string): Promise<number>;
  calculateProductionCost(cycleId: string): Promise<Money>;
}

interface SalesAnalytics {
  calculateRevenue(startDate: Date, endDate: Date): Promise<Money>;
}

```

```

    calculateAverageOrderValue(): Promise<Money>;
    getTopProducts(limit: number): Promise<Array<{productId: string, revenue:
Money}>>;
}

interface InventoryAnalytics {
    calculateTurnoverRate(productType: ProductType): Promise<number>;
    identifySlowMovingItems(thresholdDays: number): Promise<InventoryItem[]>;
    calculateWastePercentage(): Promise<number>;
}

// Analytics service
class AnalyticsService implements ProductionAnalytics, SalesAnalytics,
InventoryAnalytics {
    constructor(
        private productionRepository: ProductionRepository,
        private orderRepository: OrderRepository,
        private inventoryRepository: InventoryRepository
    ) {}

    async calculateMortalityRate(cycleId: string): Promise<number> {
        const cycle = await this.productionRepository.findById(cycleId);
        if (!cycle) throw new Error('Cycle not found');

        const totalBirds = cycle.getTotalBirds();
        const totalMortality = cycle.getDailyLogs().reduce((total, log) => total +
log.getMortalityCount(), 0);

        if (totalBirds === 0) return 0;
        return (totalMortality / totalBirds) * 100;
    }

    async calculateFeedConversionRatio(cycleId: string): Promise<number> {
        const cycle = await this.productionRepository.findById(cycleId);
        if (!cycle) throw new Error('Cycle not found');

        const totalFeed = cycle.getDailyLogs().reduce((total, log) => total +
log.getFeedConsumed(), 0);
        const totalWeightGain = cycle.getTotalBirds() * 2.5; // Estimated average
weight gain

        if (totalWeightGain === 0) return 0;
        return totalFeed / totalWeightGain;
    }

    async calculateProductionCost(cycleId: string): Promise<Money> {
        // Implementation for production cost calculation
        return new Money(0);
    }

    async calculateRevenue(startDate: Date, endDate: Date): Promise<Money> {
        // Implementation for revenue calculation
        return new Money(0);
    }
}

```

```

}

async calculateAverageOrderValue(): Promise<Money> {
  // Implementation for average order value
  return new Money(0);
}

async getTopProducts(limit: number): Promise<Array<{productId: string,
revenue: Money}>> {
  // Implementation for top products
  return [];
}

async calculateTurnoverRate(productType: ProductType): Promise<number> {
  // Implementation for inventory turnover
  return 0;
}

async identifySlowMovingItems(thresholdDays: number):
Promise<InventoryItem[]> {
  // Implementation for slow moving items
  return [];
}

async calculateWastePercentage(): Promise<number> {
  // Implementation for waste calculation
  return 0;
}
}

```

8. Compliance Management Module

8.1 Compliance Domain

```

// Compliance check enumeration
enum ComplianceCheckType {
  FOOD_SAFETY = 'food_safety',
  QUALITY_CONTROL = 'quality_control',
  SANITATION = 'sanitation',
  TEMPERATURE = 'temperature'
}

// Compliance check result
class ComplianceCheck extends Entity {
  constructor(
    id: string,
    private checkType: ComplianceCheckType,
    private batchId: string,
    private performedBy: string,
    private result: 'pass' | 'fail',
  ) {}
}

```

```

    private notes?: string,
    private correctiveActions?: string[]
  ) {
    super(id);
  }

  getCheckType(): ComplianceCheckType {
    return this.checkType;
  }

  getResult(): string {
    return this.result;
  }

  requiresCorrectiveAction(): boolean {
    return this.result === 'fail' && (!this.correctiveActions ||
this.correctiveActions.length === 0);
  }

  addCorrectiveAction(action: string): void {
    if (!this.correctiveActions) {
      this.correctiveActions = [];
    }
    this.correctiveActions.push(action);
    this.updateTimestamp();
  }
}

// Certificate entity
class Certificate extends Entity {
  constructor(
    id: string,
    private certificateNumber: string,
    private batchId: string,
    private issueDate: Date,
    private expiryDate: Date,
    private issuingAuthority: string,
    private complianceChecks: ComplianceCheck[]
  ) {
    super(id);
  }

  isValid(): boolean {
    return new Date() <= this.expiryDate &&
      this.complianceChecks.every(check => check.getResult() === 'pass');
  }

  daysUntilExpiry(): number {
    const diff = this.expiryDate.getTime() - new Date().getTime();
    return Math.floor(diff / (1000 * 3600 * 24));
  }
}

```

8.2 Compliance Services

```
// Compliance operations interface
interface ComplianceOperations {
  performCheck(checkData: any): Promise<ComplianceCheck>;
  generateCertificate(batchId: string): Promise<Certificate>;
  generateComplianceReport(startDate: Date, endDate: Date): Promise<any>;
}

// Compliance service
class ComplianceService implements ComplianceOperations {
  constructor(
    private complianceRepository: ComplianceRepository,
    private notificationService: NotificationService
  ) {}

  async performCheck(checkData: any): Promise<ComplianceCheck> {
    const check = new ComplianceCheck(
      'check-id',
      checkData.checkType,
      checkData.batchId,
      checkData.performedBy,
      checkData.result,
      checkData.notes
    );

    await this.complianceRepository.saveCheck(check);

    // Alert if check failed
    if (check.getResult() === 'fail') {
      await this.notificationService.sendAlert(
        'Compliance check failed',
        `Check type: ${check.getCheckType()}, Batch: ${checkData.batchId}`
      );
    }

    return check;
  }

  async generateCertificate(batchId: string): Promise<Certificate> {
    const checks = await this.complianceRepository.getChecksForBatch(batchId);

    // Validate all checks passed
    const failedChecks = checks.filter(check => check.getResult() === 'fail');
    if (failedChecks.length > 0) {
      throw new Error('Cannot generate certificate: compliance checks failed');
    }

    const certificate = new Certificate(
      'cert-id',
```



```

        `CERT-${Date.now()}`,
        batchId,
        new Date(),
        new Date(Date.now() + 30 * 24 * 60 * 60 * 1000), // 30 days expiry
        'Bohloko Family Farm',
        checks
    );

    await this.complianceRepository.saveCertificate(certificate);
    return certificate;
}

async generateComplianceReport(startDate: Date, endDate: Date): Promise<any>
{
    const checks = await this.complianceRepository.getChecksInPeriod(startDate,
endDate);

    const report = {
        period: { startDate, endDate },
        totalChecks: checks.length,
        passedChecks: checks.filter(c => c.getResult() === 'pass').length,
        failedChecks: checks.filter(c => c.getResult() === 'fail').length,
        passRate: checks.length > 0 ?
            (checks.filter(c => c.getResult() === 'pass').length / checks.length) *
100 : 0,
        checksByType: this.groupChecksByType(checks),
        commonIssues: this.identifyCommonIssues(checks)
    };

    return report;
}

private groupChecksByType(checks: ComplianceCheck[]): Record<string, number>
{
    // Implementation for grouping checks by type
    return {};
}

private identifyCommonIssues(checks: ComplianceCheck[]): string[] {
    // Implementation for identifying common issues
    return [];
}
}

interface ComplianceRepository {
    saveCheck(check: ComplianceCheck): Promise<void>;
    saveCertificate(certificate: Certificate): Promise<void>;
    getChecksForBatch(batchId: string): Promise<ComplianceCheck[]>;
    getChecksInPeriod(startDate: Date, endDate: Date):
Promise<ComplianceCheck[]>;
}

```

9. SOLID Principles Applied

9.1 Single Responsibility Principle (SRP)

- **Entity**: Manages only entity properties and timestamps
- **Money**: Handles only currency operations
- **UserService**: Manages only user-related operations
- **ProductionService**: Handles only production operations
- Each repository has a single responsibility for data access

9.2 Open/Closed Principle (OCP)

- **User** class is open for extension (**FarmStaffUser**, **CustomerUser**) but closed for modification
- **Authenticator** interface allows new authentication methods without changing existing code
- **ProductionOperations** interface allows new production features without modifying existing services

9.3 Liskov Substitution Principle (LSP)

- **FarmStaffUser** and **CustomerUser** can substitute **User** without breaking functionality
- All concrete services implement their interfaces correctly
- Value objects can be substituted without affecting system behavior

9.4 Interface Segregation Principle (ISP)

- Separate interfaces for **Authenticator**, **PasswordHasher**, **ProductionOperations**, etc.
- **AnalyticsService** implements multiple focused interfaces instead of one large interface
- Clients depend only on interfaces they use

9.5 Dependency Inversion Principle (DIP)

- High-level modules depend on abstractions (interfaces)
- **UserService** depends on **UserRepository**, **Authenticator**, **PasswordHasher** interfaces
- **ProductionService** depends on **ProductionRepository** and **NotificationService** interfaces
- Dependency injection through constructors

10. Conclusion

This class design provides a robust, maintainable architecture for the Bohloko Family Farm Poultry Processing System that:

1. **Follows SOLID principles** for clean, maintainable code
2. **Separates concerns** through clear module boundaries
3. **Uses domain-driven design** with meaningful entities and value objects
4. **Supports extensibility** through interfaces and abstractions
5. **Enables testability** through dependency injection
6. **Provides clear separation** between domain logic and infrastructure

The design can be implemented incrementally, starting with core entities and gradually adding services and repositories as needed.